Online Training

Session 3: Parallelism

Choosing processes, threads, CPUs and nodes on ilifu

8 September 2026

Dane Kennedy · ilifu Bioinformatics Support
University of Cape Town

View on github.io

printable version

By the end of this session…

…you will be able to:

- distinguish processes, threads, tasks, CPUs and nodes;
- request resources for serial, threaded, MPI and hybrid programs;
- measure speedup and recognise communication overhead; and
- decide when (and when not) to use multiple nodes on ilifu.

ilifu Components (review)

Login node

- Where you land when you log in (also known as "head node")
- Run Slurm commands / submit jobs, but **not** software or heavy processes

Compute nodes

- Where your processes run (also known as "worker nodes")
- Access software via modules / Singularity containers
- 1 node = 32 CPUs, ~240 GiB RAM (most Main partition nodes)

Storage

- Permanent distributed storage (CephFS), accessible from all nodes
- Your home directory and project directories live here

Network

- Gigabit Ethernet connects all nodes: high bandwidth, but **not low latency — no InfiniBand**
- This limits multi-node scaling for communication-heavy jobs (more on this later)

> **Partitions:** Main is the default; HighMem and GPU exist for bigger memory and GPU work. Check live state with `sinfo` — full node specifications are in the reference table at the end of this presentation.

What is Parallelism?

Parallel computing performs multiple parts of a computation at the same time.

It helps only when the work **can be divided** and the cost of **coordinating the workers** is small enough.

- A cluster includes many connected nodes, each with its own RAM & CPUs
- A node = single computer / server / VM / machine / box
- The work is partitioned into smaller units of work, often by splitting the dataset

Parallelism (Wikipedia)

Serial vs Parallel — the serial baseline

One step at a time

```
1 grades = [50, 65, 82, 79, 48, 72]
2 total = 0
3 for grade in grades:
4     total = total + grade
5 average = total / len(grades)
```

All instructions run sequentially on a single CPU.

Execution trace

```
grades = [50, 65, 82, 79, 48, 72]
total = 0
total = 0 + 50 = 50
total = 50 + 65 = 115
total = 115 + 82 = 197
total = 197 + 79 = 276
total = 276 + 48 = 324
total = 324 + 72 = 396
average = 396 / 6 = 66
```

Serial vs Parallel — split, process, combine

Each worker sums its chunk

```
grades = [50, 65, 82, 79, 48, 72]
chunk = len(grades) // N  # choose N=3 workers
grades_part = grades[worker_id*chunk:(worker_id+1)*chunk]
partial_sum = sum(grades_part)
average = sum(all_partial_sums) / len(grades)
```

All 3 workers process their chunk simultaneously; one worker combines the partial sums.

Execution trace

```
grades = [50, 65, 82, 79, 48, 72]
# Split: each worker takes 2 consecutive grades
W0: grades_part = [50, 65]
W1: grades_part = [82, 79]
W2: grades_part = [48, 72]
# Process: all workers sum simultaneously
W0: partial_sum = 115
W1: partial_sum = 161
W2: partial_sum = 120
# Combine (W0):
average = (115 + 161 + 120) / 6 = 66
```

Split data
Divide into N chunks

→

Process simultaneously
N workers in parallel

→

Combine results
Reduce / gather

Divide-and-conquer algorithm

More resources are not enough

Parallelism only pays off when *all* of these hold:

- **Hardware** must provide concurrent resources — multiple CPU cores (some nodes also have GPUs)
- **Software** must expose parallel work — code written with MPI, threading or OpenMP in mind
- **Coordination** has a cost — splitting work, communicating, combining results all take time
- **The workload** must be large enough to repay that cost

> Running ordinary serial software on a node with more cores will **not** make it faster — the software has to be written (and configured) to use them.

Single-node vs Multi-node Parallelism

Single-node (shared memory)

- Distributes work over many CPUs *within one node*
- Threads / OpenMP — workers share the node's memory directly
- MPI also works within one node (ranks communicate via shared memory)
- GPU acceleration — a separate execution model, also single-node

Node (32 CPUs)

Core 1 · Core 2 · … · Core 32

shared memory — fast communication

Multi-node (distributed memory)

- Distributes work over many tasks across 2+ nodes
- Requires MPI: `mpirun`, `srun`, `mpicasa` (CASA)
- Hybrid parallelism: MPI + OpenMP / MPI + GPU

Node A
Tasks 1–4

⟷ network

Node B
Tasks 5–8

⚠ **ilifu does not have InfiniBand.** Network latency between nodes is higher than at e.g. the CHPC. For communication-heavy workloads, multi-node MPI adds overhead that can make jobs *slower* than single-node — the live demo will show this.

Parallelism in Slurm

Type	`--nodes`	`--ntasks`	`--cpus-per-task`	Application setting
Serial	1	1	1	None — the default
OpenMP / Threading	1	1	8	`OMP_NUM_THREADS=8` or e.g. `--threads 8`
MPI (single-node)	1	8	1	`mpirun` launches 8 ranks
MPI (multi-node)	2	8 (4 per node)	1	`mpirun` — cross-node overhead; benchmark first
Hybrid MPI+OpenMP	2	2 (1 per node)	4	`mpirun` + `OMP_NUM_THREADS=4`

Tip: On ilifu, keep `--nodes=1` when the job fits on one node — communication can then use shared memory rather than the inter-node network. Use multiple nodes when you need more cores or memory than one node provides, and benchmark the difference.

Key Slurm Parameters (reference)

```
#SBATCH --nodes=                # number of nodes allocated
#SBATCH --ntasks=               # total tasks (normally MPI ranks)
#SBATCH --ntasks-per-node=      # tasks placed on each node
#SBATCH --cpus-per-task=        # CPU cores allocated to each task
#SBATCH --mem=                  # memory per allocated node
#SBATCH --mem-per-cpu=          # memory per allocated CPU
```

> Per node: `tasks × cpus-per-task` must fit within the node's CPUs (32 on most Main nodes).

Parameter	What it requests	Typical value
`--nodes`	Compute nodes allocated	1 unless multi-node MPI
`--ntasks`	Total Slurm tasks (MPI ranks)	e.g. 8 for 8 ranks
`--ntasks-per-node`	Tasks on each node	with `--nodes` to control layout
`--cpus-per-task`	CPU cores per task	≥ 2 for OpenMP / hybrid
`--mem`	RAM per node	Total = nodes × `--mem`
`--mem-per-cpu`	RAM per CPU	Alternative to `--mem`

Slurm parameters docs

Slurm — Serial and Multi-CPU Jobs

Serial job (1 CPU)

```
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=1

python myscript.py
```

OpenMP / threading job (8 CPUs)

```
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=8
#SBATCH --mem-per-cpu=1GB

export OMP_NUM_THREADS="$SLURM_CPUS_PER_TASK"
./my_threaded_program
```

When to use each

- **Serial:** code has no internal parallelism — increasing CPUs or nodes will *not* help
- **Threaded:** software runs multiple threads on one node — configure it with the control it supports: `OMP_NUM_THREADS` for OpenMP code, a program option like `bwa -t 8` / `STAR --runThreadN 8`, or a library setting like `OPENBLAS_NUM_THREADS` for NumPy's BLAS backend

Watch out for

- Always set the thread count explicitly — some software defaults to *all* cores on the node, not just your allocation
- Most Main nodes have 32 CPUs — and the maximum will not always give maximum speedup; test with fewer cores first

> These snippets show only the resource layout — a complete script also needs `--time`, `--mem`, `--output` / `--error` and a job name, as covered in Session 1.

Slurm — Multi-task (MPI) Jobs

Using `--ntasks` (Slurm allocates nodes)

```
#SBATCH --ntasks=8
#SBATCH --cpus-per-task=1
#SBATCH --mem-per-cpu=1GB

module add openmpi/5.0.3
mpirun python simple_mpi.py
```

Using `--ntasks-per-node` (you control layout)

```
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=8
#SBATCH --cpus-per-task=1
#SBATCH --mem=8GB

module add openmpi/5.0.3
mpirun python simple_mpi.py
```

Key differences

- **`--ntasks=8`** — Slurm decides how to spread 8 tasks across nodes; may use multiple nodes
- **`--ntasks-per-node=8`** — you specify exactly how many tasks per node; set `--nodes` too
- `mpirun` reads the Slurm allocation — no `-np` needed (`srun` can also launch MPI programs)

On ilifu

- Prefer `--nodes=1 --ntasks-per-node=8` to stay on one node
- Avoids accidental multi-node allocation that adds latency
- Tasks × CPUs per task must fit the node (32 CPUs on most Main nodes)

Slurm — Multi-node and Hybrid Jobs

Multi-node MPI

```
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=1
#SBATCH --mem=4GB

module add openmpi/5.0.3
mpirun python simple_mpi.py
```

2 nodes × 4 tasks each = 8 MPI ranks.

`--mem` is *per node*, so 2 × 4 GB total RAM.

Hybrid MPI + OpenMP

```
#SBATCH --nodes=2
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=4

module add openmpi/5.0.3
export OMP_NUM_THREADS="$SLURM_CPUS_PER_TASK"
mpirun ./hybrid_sum
```

2 nodes × 1 rank × 4 OpenMP threads = 8 cores in total.

> **Remember:** Inter-node communication is slower than intra-node on ilifu. Always benchmark before scaling to multiple nodes.

💻 Live Demo — make a prediction

Same program (`simple_mpi.py`), same total work — four different layouts:

Run	Script	Layout	Your prediction?
Baseline	`1_cpu_1_node.sbatch`	1 node × 1 rank	reference time
Same node	`4_cpus_1_node.sbatch`	1 node × 4 ranks	?
Same node	`8_cpus_1_node.sbatch`	1 node × 8 ranks	?
Two nodes	`8_cpus_2_nodes.sbatch`	2 nodes × 4 ranks	?

> **Before we run:** rank the four runs from fastest to slowest. Will 4 ranks be 4× faster? Will the two-node run beat the one-node run with the same 8 ranks?

Demo Repository

What we measured

Run	Average time	Speedup (T_1 / T_p)	Efficiency (speedup / p)
1 rank, 1 node	~0.76 s	1×	100%
4 ranks, 1 node	~0.25 s	~3×	~76%
8 ranks, 1 node	~0.155 s	~4.9×	~61%
8 ranks, 2 nodes	~0.173 s	~4.4×	~55%

Same 8 ranks, split over two nodes → slower. The work didn't change; the communication path did. And even on one node, speedup is sub-linear: splitting, coordinating and combining all cost time.

Full results in the tutorial README

Best Practices for Parallel Jobs

✓ Do

- Confirm the software actually supports the parallel model before requesting CPUs
- Start on one node and benchmark several task / thread counts
- Keep tasks × CPUs-per-task within one node's CPUs where possible
- Measure speedup and resource efficiency (`seff` — next tutorial)
- Use multiple nodes only when memory, CPU demand or measured scaling justify it
- Use `sbatch` for unattended or long-running work

✗ Don't

- Assume more CPUs automatically means faster — test it
- Spread communication-heavy jobs across nodes without benchmarking first
- Substantially over-request CPUs, memory or run time — request what a measured test run needs, plus a margin
- Rely on interactive sessions for long jobs — interactive `srun` / `sinteractive` is for short testing and debugging (Session 2)

> General cluster etiquette — login-node use, data transfers, storage cleanup — is covered in Session 1 and the Best Practices docs.

Best Practices docs

Reference: current cluster resources

Partition	Node names	Default CPUs	Max CPUs	Default Memory (GiB)	Max Memory (GiB)	Default wall-time	Max wall-time
Main	compute-[002-021]	1	32	3	232	3 hours	14 days
Main	compute-[101-105]	1	48	3	232	3 hours	14 days
Main	compute-[201-260]	1	32	3	251	3 hours	14 days
HighMem	highmem-[001-002]	1	32	15	503	3 hours	14 days
HighMem	highmem-003	1	96	15	1508	3 hours	14 days
HighMem	highmem-[004-007]	1	32	15	503	3 hours	14 days
HighMem	highmem-008	1	32	15	1007	3 hours	14 days
GPU	gpu-[001-004]	1	32	7	232	3 hours	3 days
GPU	gpu-005	1	24	7	232	3 hours	3 days
GPU	gpu-006	1	48	7	354	3 hours	3 days
GPU	gpu-007	1	48	7	354	3 hours	3 days
Devel	compute-001	1	32	-	-	3 hours	5 days

Default memory scales with the number of CPUs allocated. Jobs on the Devel partition cannot allocate memory. The current version of this table is always in the ilifu documentation.

Maximum Allocation

You can now…

- distinguish processes, threads, tasks, CPUs and nodes;
- choose a Slurm layout that matches your software;
- compare single-node and multi-node scaling; and
- explain why more resources do not guarantee proportional speedup.



Contact Support

support@ilifu.ac.za



Documentation

docs.ilifu.ac.za

Try it yourself: Tutorial 1 — Parallel Scaling